

Building a single clean-room voice model that can outcompete Rumik, Sarvam, Hume and ElevenLabs

Executive summary

As of 14 May 2026, the voice AI market is still fragmented rather than solved. Rumik's Silk 1 beta is publicly positioned as an expressive companion-focused TTS system for Hinglish, Tanglish and related code-switched use cases, with ~300 ms average latency and more than 100k user minutes per day in its own product. Sarvam's Bulbul v3 is publicly positioned as Indian-language TTS with 30+ voices, streaming APIs, code-mixed text handling and number normalisation. Hume's current stack spans Octave 2 for TTS, which Hume says can return responses in under 200 ms in 11 languages, and EVI 4-mini for multilingual speech-to-speech, which Hume says improves latency by roughly 100 ms per response over EVI 3 while requiring a supplemental LLM. ElevenLabs still splits its proposition across Flash v2.5 for ultra-low-latency synthesis at ~75 ms model latency in 32 languages and Eleven v3 for its richest expressiveness across 70+ languages, while its Text to Dialogue product is explicitly not intended for real-time conversational agents. ¹

That fragmentation is the opportunity. Based on current public documentation, no single incumbent publicly owns all of the following at once: sub-100 ms model-side responsiveness, true speech-to-speech interaction, interruptibility, deep emotional control, short-reference voice personalisation, strong multilingual coverage, and robust Indic code-switching. ElevenLabs still separates low latency from maximum expressiveness; Hume is strongest on emotional speech interaction but is still evolving its multilingual and model-composition story; Sarvam is strong on Indian-language TTS; and Rumik is focused on companion-style code-switched expressiveness rather than a broad benchmark-led platform story. ²

The most plausible path to a decisive challenger is therefore not a classic ASR → LLM → TTS pipeline. It is a unified speech-language runtime with a modern neural audio codec, duplex conversation handling, a semantic-first then acoustic refinement generator, incremental low-look-ahead onset, and a benchmark-first training loop. The open literature already exposes most of the ingredients separately: Moshi shows why speech-to-speech and full-duplex interaction reduce latency and preserve non-linguistic cues; Sesame CSM shows a practical semantic-backbone-plus-audio-decoder design; Fish-Speech S2 Pro shows a dual-autoregressive architecture with RL alignment; VoXstream shows how to begin speaking from the first word with extremely low onset delay; and EmoVoice shows that fine-grained freestyle emotion prompting can work in TTS. ³

The harder truth is that "all benchmarks" is not a meaningful goal until you define a benchmark bundle. Rumik's own public write-up says expressive TTS evaluation is "messy" and that there is "no benchmark" equivalent to the well-known LLM suites; meanwhile VoiceBench targets voice assistants, EmergentTTS-Eval targets nuanced TTS prosody and linguistic challenges, Seed-TTS eval targets zero-shot cloning metrics, Speech Arena uses crowdsourced blind A/B ranking, and ITU MUSHRA remains the standard subjective method for intermediate audio quality. The real task is to build a benchmark programme that reflects the work you actually want to win. ⁴

What beating each incumbent actually means

ElevenLabs

To beat ElevenLabs, you need to collapse its public trade-off into one runtime. ElevenLabs' own documentation positions Flash v2.5 as the fast, affordable model with ~75 ms latency in 32 languages, while positioning Eleven v3 as the most emotionally rich and expressive model across 70+ languages. Its dialogue generation capability lives on Eleven v3 and is explicitly described as not intended for real-time conversational agents, and the Flash documentation notes that number, date and currency normalisation is disabled by default to preserve latency. In other words, a challenger has to deliver Flash-class responsiveness without giving up v3-class expressiveness or language understanding. ⁵

Hume

To beat Hume, you need to match emotional intelligence and speech nativeness rather than merely TTS quality. Hume's EVI is a speech-language model that measures tune, rhythm and timbre, uses prosody for end-of-turn detection, is always interruptible, and in EVI 3 was reported by Hume as capable of under-300 ms responses on state-of-the-art hardware. Hume's latest public product line adds Octave 2 with under-200 ms TTS in 11 languages, instant cloning from a 15-second audio recording, cross-lingual accent prediction, and EVI 4-mini, which is multilingual and about 100 ms faster per response than EVI 3, but currently requires an external LLM for language generation. The opportunity here is not to beat Hume by copying its positioning; it is to produce a model that keeps Hume-grade emotional turn-taking while being stronger on code-mixed Indic speech and more self-contained in multilingual generation. ⁶

Sarvam

To beat Sarvam, you need to retain Indian-language and code-mix competence while moving beyond TTS-first product boundaries. Sarvam's public Bulbul v3 documentation describes a model built for Indian languages and accents with 30+ voices, streaming support, Hinglish/code-mixed input handling, number normalisation and natural prosody out of the box. Sarvam's Bulbul v3 blog also frames its core value around an LLM that infers emphasis, pauses, tone and pacing from text. That is a strong position for Indian-language TTS, but the public documentation is still fundamentally TTS-centric. A clean-room challenger can therefore aim to combine Sarvam's Indian-language pragmatism with Hume-like speech-to-speech interaction and Rumik-like companion expressiveness. ⁷

Rumik

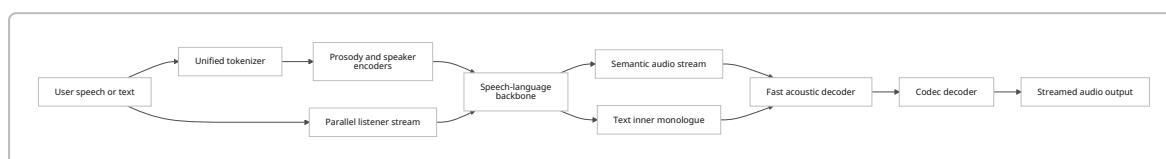
To beat Rumik, you need to win the part of the market most others still underserve: expressive code-switched speech that sounds like how bilingual users actually speak. Rumik's Silk 1 beta post is explicit that existing models were not consistent on Hinglish, Tanglish, Manglish and similar patterns; that multilingual models often learned languages as "separate islands"; that Romanised and native-script switching is central; and that expressive TTS eval remains highly manual. Rumik also publicly claims ~300 ms average latency for Silk inside its own product. That means the correct response is not a general-purpose English-first stack with a Hindi adapter. It is a phoneme-anchored, transliteration-aware, code-switch-native model trained on data that reflects how users actually mix scripts, accents, pauses and emotional tags in the same utterance. ⁸

The overlap between these four vendors is precisely the space a new model should target: one runtime that is speech-native like Hume, latency-aware like ElevenLabs Flash, India-native like Sarvam Bulbul and code-switch/emotion-native like Rumik Silk. That is difficult, but the public state of the field

suggests it is technically plausible because the ingredients now exist in open research and open code, even though no single product has assembled them all into one winner yet. ⁹

The architecture that can plausibly win

A serious challenger should be built as a single speech-language model at runtime, not as a loose three-hop stack. The reason is straightforward: Moshi's core argument is that traditional VAD/ASR/dialogue/TTS pipelines lose non-linguistic information and introduce seconds of latency, while Hume's public documentation makes the same practical point by highlighting prosody-aware turn detection, interruptibility and speech-guided response generation. If you want to beat Hume rather than just Sarvam or ElevenLabs, you need a speech-native model class. ¹⁰



The best reference design is a hybrid of several public ideas rather than devotion to one paper. Start with a Mimi-class neural audio codec because Kyutai's own explainer shows why Mimi's 12.5 frames-per-second representation, semantic token and 32-level RVQ with dropout are attractive for streaming and efficient modelling. Then place a slow semantic backbone over those tokens, with a smaller fast acoustic decoder in the style publicly described by Sesame CSM and Fish-Speech S2 Pro. That gives you the right asymmetry: a larger model spends its compute budget on meaning, prosody and high-level temporal structure, while a smaller model fills in acoustic detail. ¹¹

To win on onset latency rather than just average quality, prepend an incremental text or phoneme stream. VoXstream shows that combining an incremental phoneme transformer with a temporal transformer and depth transformer can hit 102 ms initial delay on GPU while speaking from the first word. Moshi's "inner monologue" idea, where time-aligned text tokens are predicted as a prefix to audio tokens, is also important because it appears to improve linguistic quality and planning. The practical synthesis is this: use a Moshi-style speech-native backbone for conversation and a VoXstream-style incremental front end so the model can begin output while the utterance is still unfolding. ¹²

For emotional control, the public literature points in one direction: do not rely on a single categorical emotion ID. Hume's stack uses streaming prosody measurements and prosody-conditioned responses; EmoVoice shows that free-form natural-language emotion prompting can improve control; and Rumik's public examples rely on inline emotional instructions inside utterances. A winning model should therefore support three layers of control simultaneously: discrete tags for predictable inference-time actuation, continuous affect dimensions for intensity and temporal shaping, and natural-language acting instructions for creative prompting. ¹³

For speaker identity and personalised voices, use a consented short-reference path with a dedicated speaker representation stack rather than hoping the speech-language backbone will learn everything by itself. ECAPA-TDNN remains a strong speaker-verification architecture, and WavLM remains a powerful universal speech representation model that captures both content and speaker information across downstream tasks. Seed-TTS eval uses WER plus speaker similarity from WavLM-based embeddings as core objective measures; Hume's public products now also emphasise short-reference cloning and cross-lingual voice transfer. In practice, that means your runtime should have a compact speaker encoder feeding controllable conditioning slots into the backbone, with explicit consent and audit controls around the reference flow. ¹⁴

Serving is not an afterthought. ElevenLabs' own docs stress that the ~75 ms figure is model inference only and that geographic proximity and streaming method materially affect end-to-end behaviour; Hume's practical latency comparison from New York to the US West Coast shows how network geography can dominate real-world experience; Fish-Speech S2 Pro explicitly highlights continuous batching, paged KV cache and CUDA graphs; and the PagedAttention paper shows why KV-cache efficiency matters for throughput at fixed latency. So the deployable system should use continuous batching, cache reuse, aggressive warm starts and regional clusters close to users. For South Asia in particular, infrastructure locality is itself a competitive weapon. ¹⁵

The real moat is data and alignment, not architecture

The public evidence is unusually clear that code-switched and Indic speech data is still a constraint. Rumik says high-quality expressive multi-Indic TTS data with proper code-switching was effectively unavailable to them, and recent papers on Hinglish and Indian speech continue to describe data scarcity, spontaneous-speech difficulty and code-switching as a major challenge. HiACC was presented as a new Hinglish corpus precisely because of a gap in available resources, while AI4Bharat's BhasaAnuvaad work frames Indian speech translation as limited by the scarcity and narrow coverage of public datasets. ¹⁶

That means the data strategy should be tiered. First, use broad multilingual public speech to stabilise the base model and codec. Secondly, build a transliteration-aware code-switch corpus where Latin-script and native-script variants of the same utterance are treated as equivalent at the phonemic level. Thirdly, collect premium expressive data from consented speakers speaking naturally, not just reading scripts, because the decisive gap against commodity TTS is timing, hesitation, humour, empathy and emotional carry across clause boundaries. Fourthly, collect pairwise human preferences at scale, because Fish-Speech's public S2 Pro write-up shows the value of GRPO-style post-training with multi-dimensional rewards, and Rumik explicitly says that manual listening and blind tests remain central in expressive TTS. ¹⁷

The tokenizer is a bigger strategic decision than most teams admit. Rumik's public post describes the core failure mode well: multilingual models often learn languages "like separate islands", whereas actual users switch mid-sentence, often in Roman script, often phonetically inconsistently, and often while keeping a stable regional accent. Sarvam's public docs show why number and code-mix normalisation matter, while ElevenLabs' Flash docs show what happens when low-latency systems compromise on normalisation. A winning challenger therefore needs a phoneme-anchored, multi-script tokenizer or text normalisation layer that collapses "main kal milunga", "मैं कल मिलूँगा", native-script numerals, dates, currency and named entities into stable training behaviour rather than treating each rendering as a separate problem. ¹⁸

Synthetic data should be used aggressively but not naively. The code-switch scarcity literature supports synthetic augmentation; Rumik's own story implies that trial-and-error on data engineering was central; and objective metrics such as UTMOS and DNSMOS are useful for triage at scale. The practical recipe is to generate code-switched dialogues, synthesise them with diverse consented voices or public/open models, back-transcribe them with strong ASR, reject anything with poor intelligibility or unstable prosody, and only then send a sample to human reviewers. Synthetic generation will help you reach coverage; human curation will prevent the model from learning brittle, overperformed or culturally off-key speech. ¹⁹

The benchmark stack you need before training

If you want to win “all benchmarks”, define the benchmark stack first and treat every vendor claim as provisional until you reproduce it. VoiceBench exists because voice assistants were being evaluated too narrowly; EmergentTTS-Eval exists because many TTS benchmarks miss nuanced prosody, foreign-word handling and complex pronunciation; Seed-TTS eval targets zero-shot cloning with WER and speaker similarity; Speech Arena uses blinded, order-randomised A/B tests with ELO updates; and ITU MUSHRA remains the formal subjective method most worth keeping in the loop. Rumik’s team arrived at the same practical conclusion from the opposite end: expressive voice evaluation is still messy and partially manual. ²⁰

I would structure the benchmark programme around six families.

- **Conversational performance.** Use VoiceBench-style evaluation plus your own barge-in and recovery tasks. Measure time from end of user speech to first audible output, interruption stop time, resume quality, end-of-turn accuracy, and continuation coherence after interruption. If the model cannot survive real conversational overlap, it does not beat Hume. ²¹
- **Naturalness and prosody.** Use MUSHRA as the primary human listening test, and track UTMOS and DNSMOS continuously as scalable proxies between formal listening rounds. Objective metrics are not enough, but they are extremely useful for regression testing. ²²
- **Expressiveness and linguistic challenge.** Run EmergentTTS-Eval or a close internal fork on emotions, paralinguistics, questions, foreign words, complex pronunciation and syntactic complexity. Also replicate Hume’s public nine-emotion understanding and expression protocol in your target languages, not just English. ²³
- **Voice identity and cloning.** Use Seed-TTS eval and a ClonEval-style protocol so you can score both neutral and emotional voice matching on held-out content. If your cloning looks good only on neutral read speech, it will fail in real product use. ²⁴
- **Indic and code-switching.** Build a private benchmark across Hinglish, Tanglish, Manglish, Kanglish and Latin/native-script variants; include numbers, dates, phone numbers, currency, addresses, named entities, switch-point stability and accent retention. Public sources strongly suggest this is where existing general models still wobble. ²⁵
- **Wild preference.** Run Speech Arena-style blind A/B tests with native listeners in your target regions. Crowd ELO is not the whole truth, but it is a useful warning system for regressions that lab metrics miss. ²⁶

The target scorecard should be chosen to beat the public field in the dimensions that matter most. I would not obsess over one synthetic leaderboard rank. I would insist on the following operating point: Flash-class or better model-side onset, Hume-class or better turn-taking, Sarvam-class or better Indian-language and number normalisation, and Rumik-class or better code-switched emotional naturalness, all validated in blinded listening. Public sources make clear that these properties are still split across vendors rather than unified inside one dominant system. ²⁷

A subtle but important part of this stack is adversarial evaluation. ElevenLabs’ own docs mention text normalisation caveats in Flash; Sarvam explicitly highlights code-mixed text and number normalisation; Hume stresses interruptibility and prosody-aware timing; and Rumik stresses script mixing and accent continuity. Those observations should become stress suites, not footnotes. Benchmark the exact things each incumbent is publicly proud of, then build mixed cases that combine them in one utterance. That is where a unified model has the best chance to differentiate. ²⁸

How I would execute the build

I would not start from scratch unless the budget and team are genuinely frontier-scale. Hume says EVI 3 draws on trillions of text tokens and millions of hours of speech; WavLM alone was trained on 94k hours; and even the best open systems like Moshi, CSM and Fish-Speech embody a great deal of solved engineering. The fastest credible route is therefore a clean-room warm start from open components, followed by heavy specialisation on the data and evaluation programme nobody else has fully unified.

29

In the first phase, I would adapt the tokenizer, codec configuration and speaker-conditioning path rather than chasing maximum model scale. The immediate goal is to stop languages behaving like “separate islands” and to get stable code-switching, number reading and expressive tags. In the second phase, I would run supervised fine-tuning on code-switched and expressive data until the model wins internal blinded tests against strong baselines on target domains. In the third phase, I would add duplex speech behaviour and interruption handling. In the fourth phase, I would introduce preference optimisation with multi-dimensional rewards covering intelligibility, emotional appropriateness, speaker similarity and user preference, following the direction publicly described by Fish-Speech. 30

I would also separate model milestones from product milestones. A model can sound brilliant in a notebook and still lose in production because cache warm-up, jitter, region placement, or browser audio handling ruins perceived speed. ElevenLabs explicitly notes that inference figures exclude network effects, while Hume’s practical latency example makes clear that deployment topology changes user experience materially. So every training phase should terminate in a production-style test harness, not just an offline eval spreadsheet. 31

What I would not do is almost as important. I would not scrape competitor endpoints for pseudo-ground truth; I would not treat ASR WER as a proxy for emotional quality; I would not optimise solely for English and hope Hindi adapters save the day; and I would not postpone evaluation design until after the model is already trained. Those are the shortest paths to a flashy demo that loses when users compare it against real products. The public record from Rumik, VoiceBench and EmergentTTS-Eval all points in that same direction: evaluation is the steering wheel, not the rear-view mirror. 32

Safety, provenance and commercial guardrails

If the goal is to build a real company rather than a brittle stunt, cloning and provenance need to be part of the model design, not just the policy page. NIST’s synthetic-content report explicitly groups provenance, labelling, watermarking, detection and prevention as part of the risk-reduction toolkit for generative media. C2PA is the most relevant open provenance standard, and AudioSeal shows a practical route for localised, high-speed watermark detection in generated speech. None of these mechanisms is a silver bullet on its own, but together they create an auditable chain around consented voice generation. 33

That leads to a simple commercial rule-set. The model should support short-reference personalisation only for consented voices tied to explicit account controls. Every generated asset should carry provenance metadata wherever the format permits it, and every audio stream should be watermark-capable for enterprise deployments where attribution matters. Hume’s and Octave’s public products show why personalisation is commercially important; NIST, C2PA and AudioSeal show why it also needs technical guardrails. 34

The shortest truthful answer is this: do not try to beat Rumik, Sarvam, Hume and ElevenLabs with a prettier TTS model. Beat them with a single speech-native system that fuses Hume-grade conversational prosody, Eleven-class onset speed, Sarvam-grade Indic text handling and Rumik-grade code-switched emotional realism, then prove it with a benchmark bundle that includes MUSHRA, VoiceBench-style conversation tests, EmergentTTS-style expressive stress cases, Seed-TTS-style cloning evals, and a bespoke Indic/code-switch suite. That is a hard frontier problem, but the public state of the field suggests it is now an execution problem more than a missing-ideas problem. ³⁵

¹ ⁴ ⁸ ¹⁶ ¹⁸ ²⁵ ³⁰ ³² <https://rumik.ai/research/silk>
<https://rumik.ai/research/silk>

² ⁵ ²⁷ ²⁸ <https://elevenlabs.io/docs/overview/models>
<https://elevenlabs.io/docs/overview/models>

³ ⁹ ¹⁰ ¹² <https://arxiv.org/abs/2410.00037>
<https://arxiv.org/abs/2410.00037>

⁶ ¹³ ³⁵ <https://dev.hume.ai/docs/speech-to-speech-evi/overview>
<https://dev.hume.ai/docs/speech-to-speech-evi/overview>

⁷ <https://docs.sarvam.ai/api-reference-docs/api-guides-tutorials/text-to-speech/rest-api>
<https://docs.sarvam.ai/api-reference-docs/api-guides-tutorials/text-to-speech/rest-api>

¹¹ <https://kyutai.org/codec-explainer>
<https://kyutai.org/codec-explainer>

¹⁴ <https://arxiv.org/abs/2005.07143>
<https://arxiv.org/abs/2005.07143>

¹⁵ ³¹ <https://elevenlabs.io/docs/eleven-api/guides/how-to/best-practices/latency-optimization>
<https://elevenlabs.io/docs/eleven-api/guides/how-to/best-practices/latency-optimization>

¹⁷ <https://github.com/fishaudio/fish-speech>
<https://github.com/fishaudio/fish-speech>

¹⁹ https://www.isca-archive.org/interspeech_2025/biswas25_interspeech.pdf
https://www.isca-archive.org/interspeech_2025/biswas25_interspeech.pdf

²⁰ ²¹ <https://arxiv.org/abs/2410.17196>
<https://arxiv.org/abs/2410.17196>

²² https://www.itu.int/dms_pubrec/itu-r/rec/bs/R-REC-BS.1534-1-200301-S%21%21PDF-E.pdf
https://www.itu.int/dms_pubrec/itu-r/rec/bs/R-REC-BS.1534-1-200301-S%21%21PDF-E.pdf

²³ <https://github.com/boson-ai/EmergentTTS-Eval-public>
<https://github.com/boson-ai/EmergentTTS-Eval-public>

²⁴ <https://github.com/BytedanceSpeech/seed-tts-eval>
<https://github.com/BytedanceSpeech/seed-tts-eval>

²⁶ <https://www.speecharena.org/>
<https://www.speecharena.org/>

²⁹ <https://www.hume.ai/blog/announcing-evi-3-api>
<https://www.hume.ai/blog/announcing-evi-3-api>

³³ <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-4.pdf>
<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-4.pdf>

³⁴ <https://www.hume.ai/blog/octave-2-launch>

<https://www.hume.ai/blog/octave-2-launch>